

Technical Assessment: AI-native, Spec-driven Development Pipeline

OVERVIEW

At Newton Russell, we are building AI-native engineering systems where software development is driven by structured specifications, automated reasoning, AI-assisted implementation, and deterministic governance controls.

As part of this assessment, we would like you to design and implement a prototype of an AI-native, spec-driven development pipeline. The objective is not to build a perfect production system, but to demonstrate your ability to:

- design scalable engineering workflows
- work effectively with AI-assisted development patterns
- apply software engineering fundamentals
- think critically about governance, reliability, and developer experience

OBJECTIVE

Build a prototype development pipeline where a structured feature specification can be transformed into:

- An implementation plan
- code changes
- automated tests
- validation artefacts
- deployment evidence

using AI-assisted workflows and deterministic quality controls.

REQUIREMENTS

Your solution should include the following capabilities.

1. Spec Intake

The pipeline should accept a feature specification in either:

- Markdown
- YAML
- JSON

The spec should include:

- feature objective
- user story
- business rules
- acceptance criteria

- non-functional requirements
- out-of-scope items

The pipeline should validate that required sections exist before execution begins.

2. Planning Layer

Create a planning component that converts the specification into:

- implementation tasks
- Technical Design Summary
- impacted modules/files
- risk considerations
- test strategy

This may use LLMs, rules, templates, or hybrid approaches.

3. AI-assisted Implementation

Implement a mechanism that can:

- generate or modify code from the approved specification/plan
- restrict changes to approved directories/files
- produce a summary of generated changes

You may use any framework, model provider, or orchestration approach.

4. Automated Test Generation

Generate:

- unit tests
- integration tests
- acceptance test coverage

Tests should map back to the acceptance criteria defined in the spec.

5. Quality Gates

Implement automated validation steps such as:

- linting
- type checking
- test execution
- security or policy checks

The pipeline should fail if required checks do not pass.

6. Human Approval Workflow

Introduce approval checkpoints before:

- implementation begins
- final merge/deployment

This can be lightweight but should demonstrate governance awareness.

7. Auditability

Capture:

- spec versions
- prompts or AI interactions
- generated outputs
- approvals
- validation results

The goal is to demonstrate traceability and reproducibility.

DELIVERABLES

Please provide:

1. Source code repository
2. README with setup and execution instructions
3. Example feature specification
4. Demonstration of an end-to-end run
5. Brief architectural explanation covering:
 - Design Decisions
 - trade-offs
 - limitations
 - future improvements

Optional but encouraged:

- GitHub Actions integration
- containerized setup
- agent orchestration
- evaluation metrics
- prompt/version management
- observability dashboards

EVALUATION CRITERIA

We will assess:

- system design quality
- engineering judgement
- practical use of AI tooling
- code quality

- maintainability
- governance and safety thinking
- developer experience
- clarity of communication

We are less interested in polished UI and more interested in thoughtful architecture and execution.

SUGGESTED TIMEFRAME

We expect this exercise to take approximately 6–10 hours depending on implementation depth. Please prioritise quality of thinking over breadth of features.

Deadline is 14th May 2026.

SUBMISSION

Please share:

- GitHub repository link
- setup instructions
- any relevant notes or assumptions